or delete a post.

```
php artisan make:policy PostPolicy --model=Post
```

After registering the policy, you can enforce authorization in your controllers or Blade views using methods like `authorize()` or `@can`.

13.7 Role and Permission Management

While gates and policies are powerful, larger applications often require more granular control. This is where the **Spatie Laravel Permission** package comes in. It enables role-based and permission-based access control.

You can define roles such as “admin”, “editor”, and “user”, then assign permissions to those roles. Laravel provides convenient methods to check roles and permissions in controllers and views.

```
composer require spatie/laravel-permission
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
php artisan migrate
```

Users can be assigned roles, and permissions can be granted or revoked as needed.

13.8 Exercise 39: Authentication and Authorization with Laravel 12 Breeze

13.8.1 Description

In this lab, we will implement a basic authentication and authorization system using Laravel 12’s built-in **Breeze** starter kit. Users will be able to register, log in, and access a protected profile page. The system will hash passwords using `bcrypt` and restrict access to the profile route using Laravel’s `auth` middleware. SQLite will be used as the database engine for simplicity.

13.8.2 Objectives

By the end of this lab, you will be able to:

- Scaffold authentication using Laravel Breeze
- Secure user passwords with `bcrypt`
- Create a protected route that returns the logged-in user’s profile